A Modular Framework for Interactive Visualization of ns-3 Data Center Network Simulations

Sam Lee
EPFL
sam.lee@epfl.ch

Abstract

Data center networks are an active area of research, with continuous work on congestion control, topology design, and queue-management mechanisms. However, evaluating these ideas in production-scale data centers is often difficult, making simulation tools such as ns-3 an important part of network research. In this project, we develop a modular ns-3-based visualization framework for analyzing simulated network traffic. The framework allows users to configure data center topologies, traffic workloads, link rates, queue disciplines, and congestion control schemes, and visualizes key network dynamics such as packet movement, queue occupancy, and queueing delay. The system provides a unified interface for exploring network behavior and helps users interpret congestion and queue dynamics beyond raw simulation traces.

1 Introduction

Data center networks are a central infrastructure for large-scale online services and distributed computing. Their performance depends on many interacting factors, including topology design, link capacity, traffic workload, queue management, and congestion control. As a result, researchers often use simulation to evaluate new mechanisms before deployment.

However, simulation outputs are often difficult to interpret directly. Packet traces, queue logs, and aggregate metrics can show what happened, but they do not always make it clear where congestion forms, how queues evolve, or how traffic patterns interact with the topology. This makes it difficult to build intuition from raw simulation data alone.

Visualization can make simulation results easier to interpret by presenting network behavior in an interactive and time-dependent form. However, many visualization tools are tightly coupled to a specific simulator, topology, or analysis workflow, which makes it difficult to reuse them across different configurations. This motivates a more modular framework where topology generation, simulation, trace processing, and frontend visualization are separated into extensible components.

In this project, we develop a modular visualization framework for ns-3 data center network simulations, with a design that can be extended to additional network topologies in future work. The framework supports configurable topologies, workloads, link rates, queue disciplines, and congestion-control schemes, and provides interactive visualizations of packet movement and queue dynamics.

The remainder of this report describes the underlying network model, system design, implementation details, and evaluation results.

2 Background

2.1 Link Rates and Oversubscription

Let $R_{s,t}$ denote the server-to-ToR link rate, $R_{t,a}$ the ToR-to-aggregation link rate, and $R_{a,c}$ the aggregation-to-core link rate. These rates determine the available capacity at each layer and may differ across topology levels.

The oversubscription ratio of a switch is defined as

$$\text{Oversubscription} = \frac{\text{downlink capacity}}{\text{uplink capacity}}.$$

For example, for a ToR with n attached hosts and m uplinks,

$$\text{Oversubscription}_{\text{ToR}} = \frac{nR_{s,t}}{mR_{t,a}}.$$

An oversubscription ratio greater than 1 indicates that the aggregate offered traffic from lower-layer links can exceed the available uplink capacity, creating a potential bottleneck.

2.2 Traffic Workload and Offered Load

Traffic is generated at the message level. Each workload file specifies an average message size $E[S]$ and a cumulative distribution function $F_S(s)$ over message sizes. For each generated message, the simulator samples

$$S \sim F_S.$$

The load parameter represents offered load rather than achieved throughput. In the current implementation, load is interpreted per source as a fraction of the server-to-ToR link rate:

$$R_{\text{src}} = \rho R_{s,t},$$

where $\rho \in [0, 1]$ is the configured load fraction. The total offered traffic across all hosts is therefore

$$R_{\text{total}} = N_{\text{host}} \rho R_{s,t}.$$

2.3 Poisson Message Arrivals

Each source generates messages according to an independent Poisson process. Given a per-source target rate R_{src} and average message size $E[S]$, the message arrival rate is

$$\lambda = \frac{R_{\text{src}}}{8E[S]} = \frac{\rho R_{s,t}}{8E[S]}.$$

The inter-arrival time between consecutive messages follows

$$T \sim \text{Exp}(\lambda), \qquad E[T] = \frac{1}{\lambda}.$$

For each arrival, the source samples a message size from the workload CDF and selects a random destination uniformly from all other hosts:

$$d \sim \text{Uniform}(\mathcal{H} \setminus \{s\}).$$

Figure 1 illustrates the message-size distributions used in the simulator. Each workload file contains an average message size, used to compute λ, and a message-size CDF used for sampling individual message sizes.

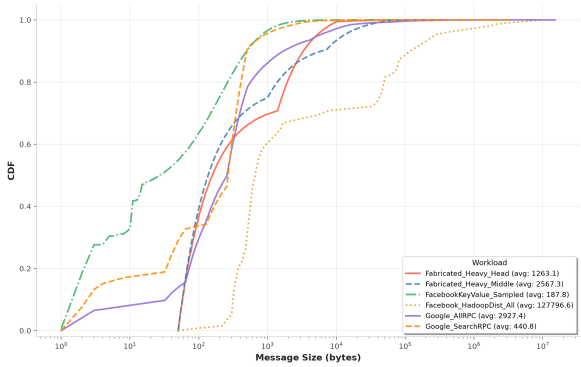

Figure 1: Message-size distributions of the supported workloads.

2.4 Queueing and Congestion Metrics

Queue occupancy serves as the primary congestion indicator. Queue capacities are derived from a bandwidth-delay product (BDP) estimate. For each directed link, the simulator estimates the worst-case round-trip time (RTT) to destinations reachable through that link. Queue capacity is then computed as

$$\text{QueueCapacity}_{\text{bytes}} = \frac{\text{linkRate} \times \text{maxRTT}}{8}.$$

RED thresholds are configured as percentages of this computed queue capacity.

The simulator records enqueue, dequeue, drop, and receive events on each link. These traces are used to compute queue occupancy, packet loss, and queueing delay. Packet-level queueing delay is defined as

$$D_i^q = t_i^{\text{dequeue}} - t_i^{\text{enqueue}}.$$

3 System Design

3.1 Architecture Overview

The proposed visualization framework consists of four stages: network simulation, data collection, data processing, and frontend rendering. Network behavior is generated in ns-3 using configurable topology, traffic, queueing, and transport parameters. During simulation, packet and queue events are recorded from each directed link and exported as structured trace data. These traces are then processed by the backend and delivered to the frontend for interactive visualization.

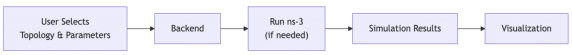

Figure 2: Overview of the system architecture.

The framework separates simulation logic from visualization logic. ns-3 is responsible for generating network events and queue dynamics, while the backend and frontend handle data delivery and interactive exploration. This modular design allows the visualization layer to evolve independently from the simulation engine.

3.2 Trace-Based Traffic and Queue Monitoring

The visualization is built from event traces collected during simulation. For each directed link, the simulator records enqueue, dequeue, drop, and receive events. These events are used to reconstruct queue occupancy, queueing delay, packet loss, and packet traversal behavior over time.

Rather than displaying raw trace files directly, the backend converts these events into visualization-ready time series and packet records. Queue occupancy is reconstructed by updating the queue state at each enqueue and dequeue event. Queueing delay is computed by matching packets between enqueue and dequeue timestamps, while drop and receive events are used to identify packet loss and successful packet traversal.

The frontend consumes the processed traces to render both topology-level and link-level views. Packet movement is visualized along links, while queue panels display time-varying metrics such as queue occupancy, packet counts, estimated queueing delay, and drop events. This trace-based design preserves the main congestion dynamics observed in the simulation while making the results easier to inspect interactively.

4 Implementation

4.1 Topology Configuration Wizard

The system provides a step-by-step configuration wizard in the frontend to simplify experiment setup. Instead of requiring direct manual editing of simulator parameters, the wizard guides users through topology selection, link configuration, queueing settings, and workload specification.

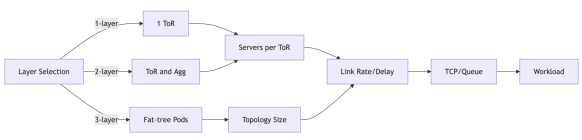

Figure 3: Configuration wizard workflow.

The wizard supports 1-layer, 2-layer, and 3-layer topologies. Depending on the selected topology, users specify parameters such as the number of servers, ToR switches, aggregation switches, pods, link rates, oversubscription ratios, queue disciplines, congestion control algorithms, and traffic workloads. The resulting configuration is translated into a simulation request and forwarded to the backend.

4.2 Topology Generation

The simulator backend is implemented in ns-3. A topology descriptor is constructed from the user-provided parameters and computes the required numbers of hosts, ToR switches, aggregation switches, and core switches.

For 3-layer configurations, the implementation follows a fat-tree structure. For 2-layer and 1-layer configurations, the corresponding switch hierarchy is generated automatically. The resulting topology is represented as an explicit link list, which is then instantiated using point-to-point channels and network devices.

Link rates are assigned according to link type, IP addresses are allocated per link, and routing tables are populated automatically to produce a complete routed topology.

4.3 Traffic Generation

Traffic generation was implemented in two stages. An initial prototype used a permutation-style traffic pattern based on `OnOffHelper`. To better model data center traffic, the final implementation replaces this approach with a per-source independent Poisson message workload generator.

Each source host generates messages according to an independent Poisson process. At every message arrival:

- a message size is sampled from the selected workload distribution,
- a destination host is selected uniformly at random among all other hosts,
- the message bytes are transmitted over TCP.

Workload distributions are stored as text files containing an average message size and a message-size CDF. During simulation, inverse transform sampling is used to generate message sizes from the selected distribution.

Unlike a short-flow model that creates a new TCP connection for every message, the implementation maintains a persistent TCP socket for each

source-destination pair. Subsequent messages between the same pair reuse the existing socket. This avoids repeated connection setup overhead and better reflects long-running communication patterns in data center workloads.

4.4 Visualization Frontend

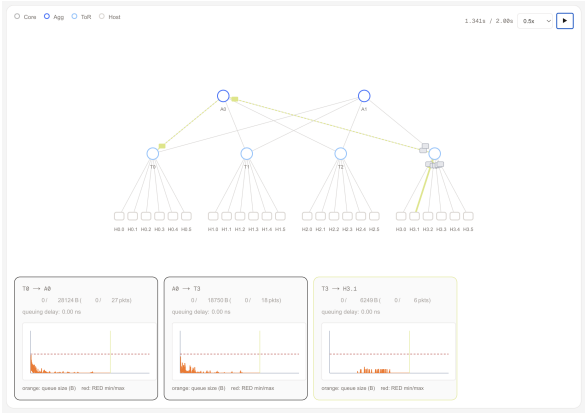

Figure 4: Screenshot of the frontend interface.

The frontend is implemented as an interactive web interface that visualizes both network topology and queue behavior. After simulation completes, packet traces are retrieved through the backend API and reconstructed into time-varying queue and packet state for visualization.

Hosts and switches are displayed as graph nodes, while links represent the network structure. Animated packet markers provide an intuitive view of traffic activity, and link appearance reflects current congestion conditions.

Users can inspect individual directed links through dedicated queue information panels. These panels display queue occupancy, packet counts, an estimated current queueing delay based on the queued bytes and link rate, and time-series graphs of queue behavior. For RED configurations, threshold values are also visualized to aid interpretation of congestion-control behavior.

The interface supports playback controls, allowing users to play, pause, and navigate through simulation time while observing congestion evolution throughout the network.

5 Evaluation

5.1 Experimental Setup

Table 1 summarizes the simulation parameters used in our evaluation.

Parameter	Value
Topology	2-layer (4 ToRs, 2 Aggs, 6 Servers/ToR)
Server-to-ToR link rate	500 Mbps
ToR oversubscription ratio	2:1
ToR-to-Agg link rate	750 Mbps
Link delay	50 μs
Congestion control algorithm	TCPDCTCP
Queue algorithm	REDQUEUE
RED thresholds	15%, 25%, 45%
Load	95%
Message dist workloads	`Google_AllRPC`, `Facebook_HadoopDist_All`

Table 1: Experimental setup used for the evaluation.

Figure 5 highlights the directed links used for queue occupancy analysis.

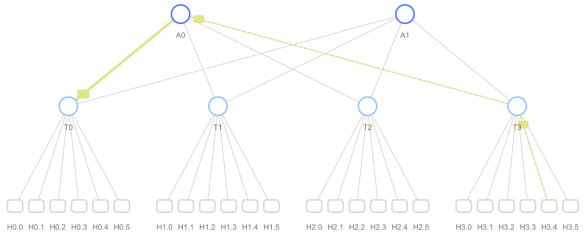

Figure 5: Topology used for the evaluation.

5.2 Impact of RED Thresholds

We first evaluate the effect of RED threshold configuration while keeping all other parameters fixed, using `Google_AllRPC` as the workload. Figure 6 compares queue occupancy along the highlighted path for RED thresholds of 15%, 25%, and 45%.

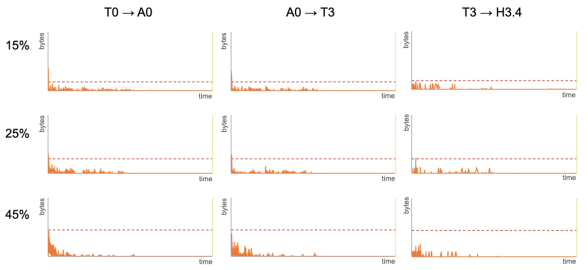

Figure 6: Queue occupancy under different RED thresholds.

A clear trend can be observed across all monitored links. Lower RED thresholds result in earlier congestion signaling, allowing DCTCP senders to react before queues grow substantially. Consequently, queue occupancy remains lower and queue buildup is limited.

As the threshold increases, congestion feedback is delayed and queues are allowed to accumulate more packets before senders reduce their transmission rates. This leads to larger transient queue

occupancies, particularly during the initial phase of the simulation.

Despite these differences, queue occupancy remains relatively shallow across all configurations. This behavior is consistent with the design goal of DCTCP, which uses ECN-based feedback to maintain low queue occupancy and avoid persistent buffer buildup.

5.3 Impact of Workload Characteristics

Figure 7 compares queue occupancy on the monitored $T0 \rightarrow A0$ uplink under `Google_AllRPC` and `Facebook_HadoopDist_All`.

Figure 7: Queue occupancy under different workload distributions.

`Google_AllRPC` is dominated by smaller messages, while `Facebook_HadoopDist_All` contains larger transfers and exhibits a heavier-tailed message-size distribution. Since both experiments target the same offered load, the smaller average message size in `Google_AllRPC` results in more frequent message arrivals.

This higher arrival frequency increases the probability that packets from multiple flows overlap on the shared $T0 \rightarrow A0$ uplink, leading to more persistent queue activity. In contrast, `Facebook_HadoopDist_All` generates fewer but larger messages. This tends to reduce the frequency of overlapping arrivals on the observed shared link, which is consistent with the less persistent queue buildup shown in the visualization.

This comparison shows that workload characteristics can change queue dynamics even when the average offered load is fixed. The visualization makes these differences easier to interpret by exposing how traffic patterns translate into queue behavior over time.

6 Future Work

Several directions for future work are possible. First, the framework currently focuses on a limited set of data center topologies. Supporting additional network architectures and routing schemes would broaden its applicability.

Second, the current workflow is trace-based and offline: the simulation completes before the generated traces are loaded into the frontend. A tighter integration between ns-3 and the frontend could enable real-time visualization of network events as the simulation runs.

Finally, future versions could provide more detailed visualization of protocol-level events and failure scenarios beyond aggregate queue and drop statistics. Examples include packet drops, retransmissions, ECN marks, duplicate ACKs, timeout events, congestion-control state transitions, and link or flow failures. Integrating these events into the visualization would make it easier to understand not only where congestion occurs, but also how transport protocols respond to failures.

7 Conclusion

This project presented a visualization framework for data center network simulations based on ns-3. The framework combines configurable topology generation, workload-driven traffic simulation, and interactive visualization of packet and queue dynamics. Through several case studies, we demonstrated how the system helps users understand the impact of congestion-control algorithms, queue-management policies, and workload characteristics on network behavior. The results suggest that visualization can provide valuable intuition beyond traditional aggregate metrics and trace analysis.